

TESTES UNITÁRIOS

14.1 INTRODUÇÃO

Neste capítulo, aprenderemos o que são testes unitários. O principal objetivo aqui é demonstrar como aplicar um teste unitário através das próprias ferramentas que o Django nos proporciona, demonstrando as grandes facilidades que existem ao utilizar sua classe principal, a `TestCase`. Vamos lá!

14.2 O QUE SÃO TESTES UNITÁRIOS

Testes unitários ou testes de unidade são basicamente testes da menor parte testável da aplicação. Se você está desenvolvendo um programa utilizando paradigma orientado a objetos, a menor parte testável do seu programa são os métodos e atributos de uma classe.

Tendo ainda como exemplo um sistema com POO, criaremos testes unitários para testarmos os métodos e atributos de nossa classe. Se nosso programa utilizasse paradigma funcional, a menor parte do programa seriam as funções, então criaríamos testes unitários para testarmos as funções do programa.

Antes de seguir, precisamos entender que não existem apenas testes unitários em uma plataforma, mas existem motivos que

veremos a seguir que fazem com eles sejam a principal escolha de um projeto. Veja a seguir outros tipos de testes que existem:

- Teste de instalação e configuração;
- Teste de integridade;
- Teste de segurança;
- Teste funcional;
- Teste de integração;
- Teste de volume;
- Teste de performance;
- Teste de usabilidade;
- Teste de regressão.

Nem todos os testes da lista são automatizados pelo próprio software e por isso podem gerar um alto custo para o projeto.

Por ser um tipo de teste que foca na aplicação de forma particionada, onde cada teste geralmente testa apenas um pedaço do código - seja um método ou atributo, no caso da programação OO, ou uma função, no caso da funcional -, ele se torna um dos tipos de testes mais baratos que existem e por isso é um dos mais implementados entre os citados na listagem anterior. A seguir temos uma imagem da pirâmide de testes.



Figura 14.1: Pirâmide de testes

Como podemos perceber, com testes unitários podemos otimizar uma grande quantidade de possíveis erros sem gerarmos um custo exorbitante para o desenvolvimento do projeto.

Sabemos que os testes de unidade precisam ser desenvolvidos pelo programador e isso vai consumir um pouco mais de tempo dele no que se refere a desenvolvimento do que caso ele não implementasse o teste. Mas no escopo global do desenvolvimento, perceberemos que o custo será menor, pois testes de unidade evitam que um programa suba para o ambiente de teste com alguns erros comuns, já que toda vez que você for lançar uma versão você testará o projeto.

Perceba que você desenvolverá o teste uma única vez e, caso um erro ocorra, você o detectará antes de subir a plataforma para homologação ou produção, desse modo será feita uma correção preventiva no programa, gerando uma grande economia de tempo e que, em longo prazo, trará grandes economias ao projeto.

Veja a seguir um gráfico que explica melhor sobre a curva de defeitos de software e perceba como os testes de unidade são importantes para otimizá-la:

Falhas do Software

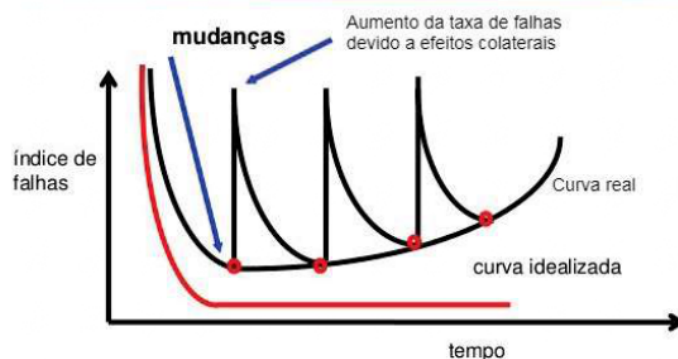


Figura 14.2: Curva de defeitos para softwares de Roger Pressman

O gráfico anterior expressa uma realidade do desenvolvimento de software muito comum, um constante aumento da taxa de falhas devido a efeitos colaterais. Esses efeitos são geralmente aquelas correções feitas em um método/função que, sem você perceber, afetam outra parte do código. Mas se você tiver um teste de unidade daquela parte do código, ao fazer a alteração no método e realizar todo o teste da aplicação, o teste vai acusar que um erro ocorreu.

Você deve estar se perguntando: mas como assim erro? Eu apenas evolui o código, modifiquei, melhorei ou qualquer coisa do tipo. Os testes de unidade, como quaisquer outros testes automatizados, não têm como função testar se o código está funcionando, mas sim verificar se, após alguma modificação naquele pedaço do código, a sua aplicação continua trazendo o resultado esperado naquele pedaço de código.

Um exemplo tosco, mas que pode refletir como queremos aqui,

é: imagine que seu método retorne um valor numérico; sabemos que todos os lugares que implementam a classe que possui esse método provavelmente precisam receber um valor numérico. Um belo dia você decide fazer com que aquele método passe a retornar uma lista, com a posição 0 sendo o valor numérico e a posição 1 sendo uma string. Sem dúvidas, a mudança ocorreu no método, mas todo o código continuará esperando um valor numérico. Quando você rodar o teste, ele vai apontar para você que é necessário mudar a forma de receber o método, pois antes você recebia um número e agora precisa receber uma lista. Isso será detectado antes de você fazer um deploy, desse modo, você conseguirá reduzir o aumento na curva de falhas do seu software.

É claro que os testes de unidade não são apenas isso. Aqui eu trouxe o mais simples dos exemplos que existem, mas, conforme veremos, existem aplicações mais avançadas para o teste de unidade. Desde já, saiba que eles representam cerca de 70% dos testes da plataforma, por serem mais baratos.

14.3 TDD

Em programação existe o que chamamos de desenvolvimento guiado por testes ou, em inglês, TDD (*Test-Driven Development*). O TDD é uma técnica de desenvolvimento que visa começar o desenvolvimento de cada funcionalidade do software pelo seu teste, e é aqui que o teste de unidade entra. Parece ser algo muito diferente, mas não é. Começar a desenvolver uma funcionalidade pelo teste facilita o desenvolvimento, porque desse modo faremos a funcionalidade com o objetivo de passar no teste que foi desenvolvido. Existem três etapas no TDD, veja cada uma delas na imagem a seguir:

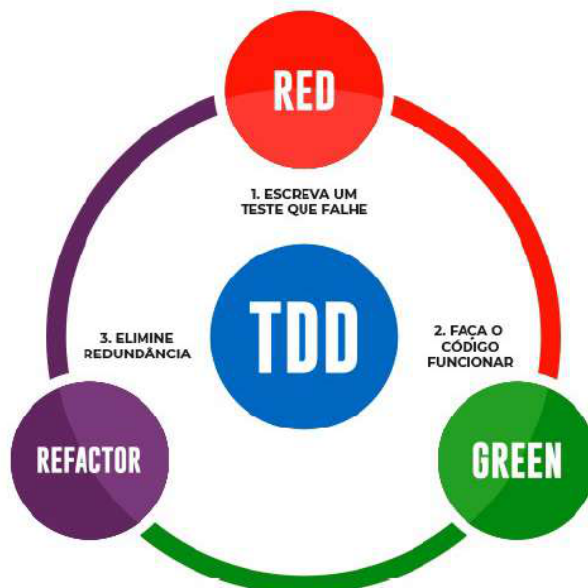


Figura 14.3: As três etapas do TDD

1. **Red (Vermelho):** escreva um pequeno teste automatizado que, ao ser executado, falhará porque ainda não temos o código da funcionalidade criado.
2. **Green (Verde):** implemente um código que seja suficiente para ser aprovado no teste recém-escrito. Perceba que aqui você terá o teste já criado na etapa Red para utilizar como base no desenvolvimento da sua funcionalidade.
3. **Refactor (Refatorar):** refatore o código a fim de ele ser melhorado, deixando-o mais funcional e mais limpo.

Vendo a aplicação das três etapas do TDD, temos em conjunto com ele o modelo *F.I.R.S.T.* que dará uma ótima orientação de

quais itens devem ser priorizados na hora de criar e executar os testes:

- **F (Fast)**: rápidos;
- **I (Isolated)**: isolados;
- **R (Repeatable)**: reproduzíveis;
- **S (Self-verifying)**: autoverificáveis;
- **T (Timely)**: oportunos, no momento certo.

Com esse modelo, conseguimos analisar com mais facilidade em quais cenários da nossa aplicação vamos trabalhar com a programação guiada por testes. Não existe uma regra que diga e exija que você tenha todo o seu código sendo desenvolvido em conjunto com testes. Utilize o modelo F.I.R.S.T. e analise quais os melhores lugares para aplicar as três etapas do TDD dentro do seu programa.

Resumindo, entre as principais vantagens do TDD está a de termos um código mais limpo e fácil de passar por uma refatoração.

14.4 TESTES UNITÁRIOS NO DJANGO

Nesta etapa, veremos alguns recursos do Django que nos permitem desenvolver testes de unidade facilmente através do módulo `django.test`, que já vem instalado no Django e implementa o módulo `unittest`, que é nativo do Python. Veremos também alguns módulos de processamento por thread, requisições HTTPS por background e outros que facilitam bastante o desenvolvimento dos testes unitários. Vamos lá:

Dentro de todo app que criamos no Django já vem um arquivo

chamado `tests.py` . Vamos apagá-lo, pois criaremos nossos testes de uma maneira mais organizada. Abra a pasta `medicSearch` e delete o arquivo `tests.py` . Agora crie dentro da pasta `medicSearch` um diretório chamado `tests` e dentro crie um arquivo chamado `__init__.py` . Com essa pasta, o diretório ficará parecido com este a seguir:

```
livro_django
├── medicSearch
│   ├── migrations
│   ├── models
│   ├── static
│   ├── templates
│   ├── tests
│   │   └── __init__.py
│   ├── urls
│   ├── views
│   ├── __init__.py
│   └── ... Arquivos ocultos para otimizar a página
└── medicSearchAdmin
    └── ... Arquivos ocultos para otimizar a página
```

14.5 PRIMEIRO TESTE UNITÁRIO

Vamos criar nosso primeiro teste de unidade no projeto, que listará um usuário do Django e, caso retorne um valor, o teste estará correto. Abra o diretório `medicSearch/tests` e crie dentro dele um arquivo chamado `test_model_user.py` . Vamos adicionar o código a seguir:

medicSearch/tests/test_model_user.py

```
from django.test import TestCase
from django.contrib.auth.models import User

class UserModelTestClass(TestCase):
    def setUp(self):
        pass
```



```
def test_user_exist(self):
    user = User.objects.first()
    self.assertIsNotNone(user)
```

Perceba que criamos a classe `UserModelTestClass` que herda de `TestCase`. `TestCase` possui diversos módulos que podemos utilizar nos testes de unidade para verificar as saídas esperadas em nossos testes. Entre eles existe o principal, o `setUp`, que veremos mais a seguir.

Realizamos a listagem de um usuário e após isso verificamos através do método `assertIsNotNone` se o resultado trazido é nulo ou não. Caso retorne algo, o teste vai passar; caso retorne um resultado nulo, o teste de unidade vai reclamar. Rode o comando `python manage.py test` em seu console e veja que receberá uma saída similar à seguinte:

```
(venv) tiago_luiz@ubuntu:~/livro_django$ python manage.py test
Creating test database for alias 'default'...
AssertionError: unexpectedly None
-----
-----
Ran 1 test in 0.002s
FAILED (failures=1)
Destroying test database for alias 'default'...
(venv) tiago_luiz@ubuntu:~/livro_django$
```

Esse teste não foi bem-sucedido e você deve estar se perguntando o motivo. Os testes unitários não utilizam e nem poderiam utilizar a mesma base de dados que utilizamos em nossa aplicação de desenvolvimento. Por isso, precisamos configurar um `Setup` inicial para cada teste unitário que criamos. Vamos ver a seguir como realizar isso. Altere o arquivo `test_model_user.py` para ficar igual ao seguinte:

medicSearch/tests/test_model_user.py

```
from django.test import TestCase
from django.contrib.auth.models import User

class UserModelTestClass(TestCase):
    def setUp(self):
        User.objects.create(username='teste.unitario', password='
123456')

    def test_user_exist(self):
        user = User.objects.first()
        self.assertIsNotNone(user)
```

Como falamos anteriormente, os testes de unidade não olham para o banco de desenvolvimento. Perceba que quando rodamos o comando `python manage.py test` a primeira linha que roda é `Creating test database for alias 'default'...` e a última é `Destroying test database for alias 'default'...`, ou seja, ao iniciar o teste, ele cria um banco para realizar os testes e depois o destrói. Desse modo, antes de realizar os testes, precisamos popular esse banco com os dados de que precisamos para realizar os testes necessários. Não é nada complicado, você não precisa abrir conexão nem nada do tipo, apenas adicione os dados necessários através do método `setUp` que é herdado da classe `TestCase`.

Se rodarmos o comando `python manage.py test` novamente, teremos uma saída diferente agora. Veja:

```
(venv) tiago_luiz@ubuntu:~/livro_django$ python manage.py test
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.
-----
-----
Ran 1 test in 0.003s
OK
```

```
Destroying test database for alias 'default'...
(venv) tiago_luiz@ubuntu:~/livro_django$
```

Já que criamos um usuário no método `setUp`, agora, quando chamamos o método `test_user_exist`, o retorno dele será o usuário criado no `setUp` e, ao verificar o retorno com o método `assertIsNotNone`, ele retornará o valor `True` e o teste passará.

Existem outros métodos do módulo `unittest` que podemos utilizar para saber se um determinado resultado está retornando o que é esperado. A seguir temos uma lista com os principais:

- **`assertFalse(self, expr, msg=None)`**: verifica se o valor do `return` é falso.
- **`assertTrue(self, expr, msg=None)`**: verifica se o valor do `return` é verdadeiro.
- **`assertEqual(self, first, second, msg=None)`**: verifica se o parâmetro `first` é igual ao parâmetro `second`.
- **`assertNotEqual(self, first, second, msg=None)`**: verifica se o parâmetro `first` é diferente do parâmetro `second`.
- **`assertListEqual(self, list1, list2, msg=None)`**: verifica se a lista do parâmetro `list1` é igual à do `list2`.
- **`assertTupleEqual(self, tuple1, tuple2, msg=None)`**: verifica se a tupla do parâmetro `tuple1` é igual à do `tuple2`.
- **`assertSetEqual(self, set1, set2, msg=None)`**: verifica se o conjunto do parâmetro `set1` é igual ao do `set2`.
- **`assertDictEqual(self, d1, d2, msg=None)`**: verifica se o dicionário do parâmetro `d1` é igual ao do `d2`.
- **`assertCountEqual(self, first, second, msg=None)`**: verifica entre dois elementos iteráveis se possuem a mesma quantidade de elementos.

- **assertIsNone(self, obj, msg=None)**: verifica se o resultado é nulo.
- **assertIsNotNone(self, obj, msg=None)**: verifica se o resultado não é nulo.
- **assertContains(self, response, text)**: verifica se no parâmetro `response` contém a string que é passada no parâmetro `text`.

Existem outros métodos, caso deseje conhecê-los, consulte a documentação do python unittest no tópico sobre os métodos asserts (<https://docs.python.org/3/library/unittest.html#assert-methods>).

14.6 USANDO O CLIENT PARA FAZER REQUISIÇÕES

Agora, veremos a classe `Client` do módulo `django.test`. Com ela, é possível realizarmos requisições HTTPS dentro dos nossos testes de unidades. Vamos criar a seguir um pequeno exemplo de `get` que retornará para nós o resultado `html` da `view` de médicos. Depois, veremos se o status de resposta é igual a 200 e se existe um determinado texto dentro do `html`.

Crie dentro da pasta `medicSearch/tests` um arquivo chamado `test_medic_view.py` e adicione o código a seguir:

medicSearch/tests/test_medic_view.py

```
from django.test import TestCase, Client
from django.contrib.auth.models import User
from medicSearch.models.Profile import Profile
from django.db import transaction, IntegrityError
```

```

class MedicViewTestClass(TestCase):
    def setUp(self):
        try:
            with transaction.atomic():
                user = User.objects.create(username='teste.unitario', password='123456')
                profile = Profile.objects.get(user=user)
                profile.role = 2
                profile.save()
        except IntegrityError as e:
            print("Erro ao criar usuário. Descrição: %s" % e)

        self.client = Client()

    def test_medics_list(self):
        response = self.client.get('/medic/')
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, 'Foram encontrados: 1 medicamento(s)')

```

Perceba que agora criamos uma `transaction` em nosso `setUp`. O padrão de um `user` inicial é ser criado como usuário, então foi necessário alterá-lo para ser um médico, pois precisávamos listar pelo menos um médico em nossa `view`. Além disso, foi preciso instanciar o método `Client`.

Se você tiver algum problema de exigência do `CSRF_TOKEN` no momento do teste, pode passar como parâmetro para a classe `Client` o valor `enforce_csrf_checks=False`. Desse modo, o Django não vai exigir que haja um `token csrf` dentro da requisição. Isso pode ser exigido se você for realizar um teste simulando um `POST` de login, por exemplo.

A classe `Client` possui todos os métodos de requisição HTTP. Veremos a seguir um exemplo real, no qual criaremos um teste com o login da nossa plataforma utilizando o método `POST`. Os demais métodos que diferem são similares ao `GET` que já

vimos e o POST que veremos a seguir:

Você deve ter visto que, em vez de utilizar uma consulta como `Profile.objects.filter(user=user).first()` , foi utilizado `Profile.objects.get(user=user)` . Esse é um modo de buscar um único resultado, onde buscamos no banco baseado em um parâmetro. Neste caso, usamos `user` , mas poderia ter sido `Profile.objects.get(pk=1)` , onde `pk` é o `id` do registro no banco de dados. Desse modo, não é preciso utilizar o método `first()` para dizer que você quer retornar apenas um objeto como resultado.

14.7 CRIANDO UM TESTE DE LOGIN

Para juntar o conhecimento adquirido, vamos criar um teste de login, que terá tudo que vimos até agora dentro deste capítulo. Crie dentro da pasta `medicSearch/tests` um arquivo chamado `test_login.py` e adicione o código a seguir:

medicSearch/tests/test_login.py

```
from django.test import TestCase, Client
from django.contrib.auth.models import User

class LoginTestClass(TestCase):
    def setUp(self):
        User.objects.create(username='teste.unitario', password='123456')
        self.client = Client()

    def test_login(self):
        response = self.client.post('/login', {
            'username': 'teste.unitario',
```

```

        'password': '123456'
    }, **{'Content-Type': 'application/x-www-form-urlencoded'
    })
    self.assertEqual(response.status_code, 200)

```

Como podemos ver, com tudo que foi falado, é possível criarmos testes de unidade para nosso programa sem dificuldade.

Perceba que no login substituímos o método `get()` pelo `post()` e, além do primeiro parâmetro que passamos do endpoint, foi passado também um segundo parâmetro, do tipo dicionário com os valores que devem ser enviados para a API de login e um terceiro, que é o que chamamos de *magic variable* em Python (nesse caso, o famoso `kwargs`, que contém um dicionário em que podemos passar as `headers` da requisição).

Através da lib `Client` é possível usar todos os métodos que representam os verbos HTTP: `get`, `post`, `put`, `patch`, `delete` etc. Todos podem se comportar da mesma maneira que vimos anteriormente com esses três parâmetros.

14.8 COBERTURA DE CÓDIGO

Antes de concluir, vamos instalar uma biblioteca que nos permitirá ver o quanto nosso código está coberto por testes de unidade. Isso é legal para termos uma ideia dos testes já feitos e os que ainda não foram criados para a plataforma. É bem útil para projetos que já existem e ainda não possuem testes de unidade e agora você precisa ter uma ideia do quanto o seu código já está coberto após começar a implementar testes de unidade.

Vamos instalar o módulo `coverage` rodando o comando `pip install coverage` :

```
(venv) tiago_luiz@ubuntu:~/livro_django$ pip install coverage
Collecting coverage
  Using cached coverage-5.2-cp38-cp38-win32.whl (206 kB)
Installing collected packages: coverage
Successfully installed coverage-5.2
(venv) tiago_luiz@ubuntu:~/livro_django$
```

Agora que foi instalado, podemos rodar o comando `coverage run manage.py test` sempre que quisermos. Ele rodará nossos testes unitários, gerará um banco de dados `sqlite` com o relatório dos testes e poderemos ver a cobertura do código a qualquer momento. Vamos ver a seguir as duas maneiras de ver a cobertura dos testes.

Para ver a cobertura do código pelo terminal, rode `coverage report --omit=*/venv/*` e terá um resultado similar ao do terminal a seguir:

```
(venv) tiago_luiz@ubuntu:~/livro_django$ coverage report
Name                                                    Stmts   Mi
ss Cover
-----
-----
manage.py                                              12
  2    83%
medicSearchAdmin\__init__.py                           0
  0   100%
medicSearchAdmin\settings\__init__.py                  0
  0   100%
medicSearch\views\AuthView.py                        100
72    28%
medicSearch\views\HomeView.py                          3
  1    67%
medicSearch\views\ProfileView.py                      42
34    19%
medicSearch\views\__init__.py                          3
  0   100%
```

```

-----
TOTAL                                     578      2
00      65%

```

```
(venv) tiago_luiz@ubuntu:~/livro_django$
```

Eu precisei ocultar o resultado total, porque eram muitas linhas, mas será algo parecido com isso. Você também pode gerar o resultado em um arquivo HTML para ver em seu navegador. Rode o comando `coverage html` no terminal e uma pasta chamada `htmlcov` será criada dentro do projeto. Abra o arquivo `index.html` que fica dentro da pasta e você verá uma tela igual à da imagem a seguir:

Module ↑	statements	missing	excluded	coverage
manage.py	12	2	0	83%
medicSearchAdmin__init__.py	0	0	0	100%
medicSearchAdmin\settings__init__.py	0	0	0	100%
medicSearchAdmin\settings\development.py	5	0	0	100%
medicSearchAdmin\settings\settings.py	39	0	0	100%
medicSearchAdmin\urls.py	6	0	0	100%
medicSearch__init__.py	0	0	0	100%
medicSearch\admin.py	22	2	0	91%
medicSearch\forms\AuthForm.py	12	0	0	100%
medicSearch\forms\MedicForm.py	8	0	0	100%
medicSearch\forms\UserProfileForm.py	14	0	0	100%
medicSearch\migrations\0001_initial.py	8	0	0	100%
medicSearch\migrations\0002_auto_20200225_1649.py	5	0	0	100%
medicSearch\migrations\0003_auto_20200225_1650.py	5	0	0	100%
medicSearch\migrations\0004_address_phone.py	4	0	0	100%
medicSearch\migrations\0005_auto_20200508_1335.py	4	0	0	100%
medicSearch\migrations\0006_remove_profile_status.py	4	0	0	100%
medicSearch\migrations\0007_profile_token.py	4	0	0	100%
medicSearch\migrations__init__.py	0	0	0	100%
medicSearch\models\Address.py	16	1	0	94%
medicSearch\models\City.py	9	1	0	89%
medicSearch\models\DayWeek.py	8	1	0	88%
medicSearch\models\Neighborhood.py	9	1	0	89%

Figura 14.4: HTML com porcentagem de cobertura de testes

Ao clicar em qualquer link, você verá a descrição das áreas não cobertas do código.



Figura 14.5: HTML com porcentagem de cobertura de testes

Nessa última imagem, vemos a parte do teste de login que nosso teste não cobriu. Com isso, vemos como é possível ter controle da cobertura de testes que nosso código possui.

Acostumar-se a trabalhar desenvolvendo testes antes das funcionalidades pode ser algo complicado no início, mas com o passar do tempo se tornará costume. A maioria das empresas tem exigido programadores que desenvolvam e tenham conhecimento em TDD e dentro dele principalmente os testes de unidade. Sem dúvidas, é válido aprender a trabalhar com os demais testes que existem em desenvolvimento de softwares, mas o teste de unidade é, além de um ótimo começo, uma grande *skill* que todo desenvolvedor precisa ter para se destacar no mercado.